

Pharmacovigilance Adverse Event Intake and Triage Service

Product Requirements Document and Architecture

UE24AM342AA5 - ML System Design and AgentOps - Mini Project 14 (Jackfruit, Level 3)

Part	Contents
Part 1 - PRD	Purpose, goals and non-goals, users, scope, user stories with acceptance criteria, requirements by tier, non-functional requirements, success metrics, risks, milestones, open decisions
Part 2 - Architecture	Design principles, deployment diagram with boundaries, services, boundary justification, request lifecycle, agent graph, tools, failure behaviour, retrieval design, data contracts, reliability, guardrails, observability, performance and cost hypotheses, decisions and rejected alternatives, repository layout
Part 3 - Repo addendum	A prompt for Antigravity that installs both documents into the repo as the shared source of truth

Numbers marked **[confirm]** or **hypothesis** are proposals to be confirmed or replaced by measurements. Regulatory clause numbers and deadlines are deliberately not stated here: they must come from the retrieved documents.

PART 1

Product Requirements Document (PRD)

Product: Pharmacovigilance Adverse Event Intake and Triage Service. **Course:** UE24AM342AA5, Mini Project 14 (Jackfruit, Level 3). **Status:** draft for team confirmation. Items marked [**confirm**] must be decided by the team before the phase that depends on them.

1. Purpose

Drug companies must assess every adverse event report and, depending on seriousness and expectedness, report it to the regulator within a fixed timeline. This service accepts a report, validates it, assesses seriousness and expectedness against the reference documents, and routes it to expedited, periodic or non-reportable with cited evidence. It **recommends only**. A qualified person for pharmacovigilance (QPPV) decides. It never submits a regulatory report.

2. Goals and non-goals

Goals	Non-goals
Correct routing of the reference case (section 4.1 of the brief) with citations that resolve to retrieved text	Submitting anything to a regulator or making a final determination
Escalate to a human instead of guessing when evidence is weak, timing is unclear, or the drug is outside the corpus	Model training or fine-tuning
Run as a production-style service: async API, queue, scalable workers, observability, resilience	Cloud deployment (runs locally under Docker Compose)
Every run reconstructible from its run_id	Real patient or case data (synthetic only)
Honest, measured evaluation, load and cost numbers	Covering every drug (one package insert is in scope)

3. Users

User	Need	How the product serves them
QPPV (primary decision maker)	A defensible recommendation with evidence, and a clear signal when the system is unsure	Result page shows routing, basis, citations, verification status, and an escalation reason when applicable
Case processor / intake client	Submit a report and know quickly whether it was accepted	REST intake with immediate run_id; field-level errors for bad input
Operator / developer	Know the system is healthy and diagnose failures	Health, metrics, structured logs, traces, runbook
Examiner (course)	See requirements met and judgement shown	10-item demo, report, repo, viva

4. Scope

In scope: the eight functional requirements (F1-F8) and the agent, LLMOps and platform requirements prioritised in section 6. The corpus is the NDCT Rules 2019 adverse event provisions, the IPC pharmacovigilance guidance for marketing authorisation holders, and one package insert for the chosen drug [**confirm: drug**].

Out of scope: anything under section 2 non-goals, plus UI polish beyond a clear professional client, multi-tenant auth, and multiple drugs.

5. Core user stories and acceptance criteria

ID	Story	Acceptance criteria
S1	As an intake client I submit a valid report and get a run id at once	POST returns 202 with run_id in under 1 second; GET run shows queued, running, then a terminal status
S2	As an intake client, a malformed report is rejected	422 with field-level errors; record in quarantine; queue length unchanged (no agent run started)

ID	Story	Acceptance criteria
S3	As a QPPV I see the routing and why	Result has routing, basis, seriousness criteria, expectedness match, and citations; every citation resolves to a passage retrieved in that run
S4	As a QPPV I am told when the system cannot answer	Out-of-corpus drug, unmatched reaction, or unclear timing yields status escalated with a reason, never a fabricated answer
S5	As a QPPV I can trust the citations	A verifier rejects unsupported answers; failed verification triggers another search or escalation; an unverified answer is never returned
S6	As an operator I can reconstruct a run	Given a run_id, /trace shows every tool call, query, passage, prompt version, per-step latency, cost and verification outcome
S7	As an operator I can see failure handling	With the provider down, the circuit breaker opens and runs escalate with reason 'LLM unavailable'
S8	As the reference case reporter I describe a reaction in everyday words	Lay words are mapped to the medical terms the insert uses; the reaction is judged listed or not, seriousness applied, and routing cited

6. Functional requirements and priority

Req	Summary	Tier	Acceptance
F1	REST intake against a published data contract	1	OpenAPI plus JSON schema published; S1
F2	Schema, range, category, cross-field validation; quarantine rejects	1	S2
F3	Extract drug, reaction, demographics, timing, outcome	1	Entities present in trace; timing gaps trigger escalation
F4	Retrieve listed reactions, seriousness criteria, applicable provision	1	Reference case retrieves all four passages of section 4.1
F5	Grounded seriousness and expectedness with citations	1	S3, S5
F6	Route to expedited / periodic / non-reportable with basis	1	Routing matches the reference answer
F7	Escalate when reaction unmatched, timing unclear, or drug outside corpus	1	S4
F8	Health, metrics, structured logs	1-2	/health, /metrics, JSON logs with correlation id
A1-A5	LangGraph agent, 2+ agents, verification, async API with queue, resilience	1-2	Sections 5 and 8 of the architecture
A6-A8	Agent metrics dashboard, --scale test, web client	3	Demo items 9 and client screens
L1-L4	Retrieval strategy, prompt versions, 20-scenario eval, injection defence	1-2	docs/eval_results.md; guardrail demo
L5-L6	Cost tracking, feedback and live scoring	3	agents/COST.md; feedback summary
D1-D5	Data contract, ingestion CLI, Compose (4+ services), no secrets, observability	1-2	Clean-clone test
D6-D8	SLO and Locust, caching and rate limit, runbook and incident log	2-3	docs/SLO.md committed first

7. Non-functional requirements

Area	Requirement
Latency SLO	Declared p95 for a complete run (submission to final result), measured with the stub LLM. Proposed starting point [confirm]: p95 at or below 60 s at the tested concurrency, justified by expedited timelines being counted in days while a QPPV needs results within one working session. Commit docs/SLO.md before any load test; never change it after seeing results.
Safety	Zero unverified answers returned. Zero outputs phrased as a final regulatory decision or submission.
Reliability	Timeouts on every model and tool call; bounded retry with backoff; circuit breaker; incomplete run is failed or escalated, never confident.
Observability	Structured logs with correlation id (run_id); OpenTelemetry traces to Jaeger; Prometheus metrics on every service.

Area	Requirement
Security	No credentials, endpoints or model ids in the repo; injection defence on all free-text fields; output filter; toxicity check.
Cost	Per-step token and cost accounting; rate limit per client; whole-run cache; local embeddings.
Portability	docker compose up from a clean clone; nothing runs outside containers; worker scalable to 3.
Reproducibility	Idempotent ingestion; index metadata records build time, source versions and chunk count.

8. Success metrics

Metric	Target	Source
Unverified answers returned	0 (hard requirement)	Soak test, eval
Routing accuracy on non-escalation scenarios	Report measured value; aim for all of the reference cases correct	eval/run
Correct-escalation rate on out-of-corpus / unclear scenarios	Report measured value; aim for 100%	eval/run
Groundedness, context relevance, faithfulness, hallucination rate	Report all four before and after one deliberate change	docs/eval_results.md
p95 vs declared SLO, and failing concurrency	State met or not met, with the responsible component	docs/load_results.md
Cost per 100 runs and largest driver	Report measured value	agents/COST.md

9. Constraints and assumptions

- Free tools only. The model uses a free tier with low rate limits; development and load tests use the stub LLM.
- Synthetic data only. Corpus documents are public and must be read from the real files; the system never cites text it did not retrieve.
- Team of three, one working day of build time. A working thin system beats an ambitious broken one; if the system does not start at the demo the project scores zero.
- Regulatory clause numbers and deadlines are taken from the retrieved documents, never from memory.

10. Risks and mitigations

Risk	Impact	Mitigation
Retrieval misses the provision the reference case depends on	Wrong or escalated reference case	Query rewrite tool, listed-terms chunks, synonym table, second-pass search from read text, recall@5 tracking
Free-tier rate limits or outage	Slow demo or failed runs	Stub default; breaker and escalation fallback; real model only for demo and eval
Agent loops or overspends	Latency and cost blow-up	MAX_STEPS, MAX_TOOL_CALLS, RUN_TIMEOUT_S; rate limit; cache
Plausible but fake clause numbers from the model	Regulatory misinformation	Verifier requires every citation to resolve to retrieved text; humans check the reference case against the PDFs
Integration failure between members	No working demo	Frozen contracts in shared/contracts.py; merge checkpoints at hour 4 and after Phase 3
Secrets committed by accident	Leaked key	.gitignore, pre-commit secret scan, rotate key if leaked
Index concurrency with 3 workers	Errors under scale	Read-only index volume for workers; fall back to a Chroma server container if locking appears

11. Milestones

Milestone	Definition of done
Checkpoint 1 (about hour 4)	docker compose up works from a clean clone; valid request returns a placeholder result; invalid request is quarantined

Milestone	Definition of done
Checkpoint 2 (about hour 8)	Reference case correct end to end with resolving citations; out-of-corpus case escalates
Checkpoint 3	Guardrails, breaker, traces; 20-scenario eval set validated; SLO committed
Final	Eval before/after, Locust results, README tested from a clean clone, report, all 10 demo items rehearsed

12. Open decisions [confirm]

- Which drug (a package insert that clearly lists severe skin reactions).
- The p95 SLO number, committed in docs/SLO.md before any load test.
- Vector store: Chroma (default here) or FAISS.
- Which member owns the incident log entries as failures happen.

PART 2

Architecture Document

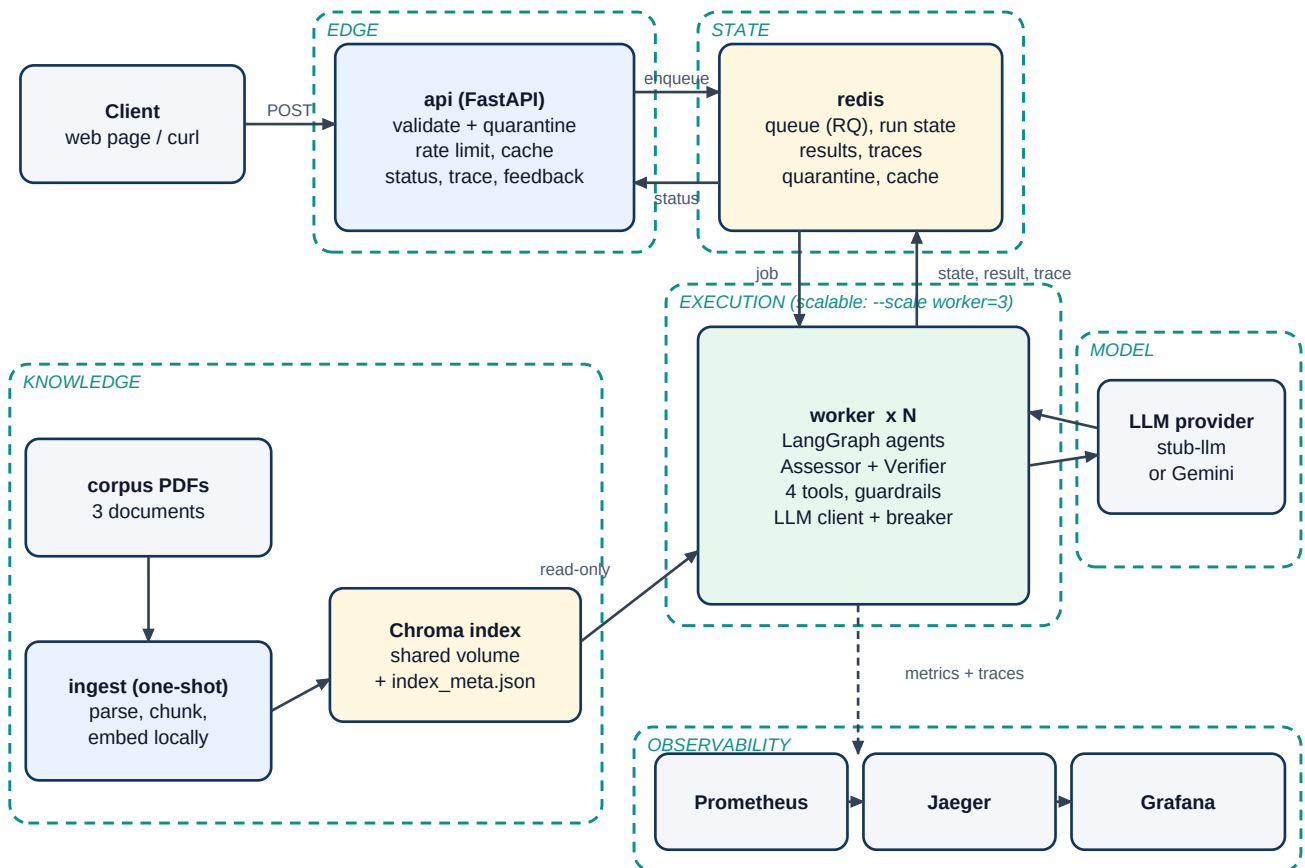
This document describes how the system is built and where the deployment boundaries sit. It is the source for the report's architecture section. Values marked **hypothesis** must be replaced by measurements.

1. Design principles

- **Fail safe.** Any doubt or incompleteness ends in failed or escalated, never a confident answer.
- **Evidence or silence.** Every claim cites text retrieved in the same run; a separate verifier checks it.
- **Validate at the door.** Bad input never starts an agent run.
- **Async by default.** Runs take tens of seconds; the API returns a run id and the caller polls.
- **Everything replaceable by config.** Model, provider, index location, limits and prices come from the environment.
- **Observable end to end.** One correlation id (run_id) ties logs, traces, metrics and stored trace together.

2. Deployment architecture

Deployment view - dashed = deployment boundary, solid = container



Dashed rectangles are deployment boundaries. Solid boxes are containers under Docker Compose (the ingest box runs once). The api also exports metrics and traces; the arrow from the worker stands for both to keep the figure readable.

3. Services and responsibilities

Service	Responsibility	Tech	Scale	State	If it fails
api	Intake, validation, quarantine, rate limit, cache lookup, status, trace, feedback, web client	FastAPI, Pydantic v2	1 (can scale)	None (stateless)	No new runs; queued and running runs continue

Service	Responsibility	Tech	Scale	State	If it fails
worker	Runs the LangGraph agents: tools, guardrails, LLM client with breaker	RQ worker, LangGraph, LangChain	1 to 3+ (--scale)	None; state kept in redis	Queued runs wait; a crashed job is marked failed
redis	Queue, run state, results, traces, quarantine, cache, rate-limit counters, feedback	Redis with append-only file	1	Yes (volume)	System stops accepting work; health shows it
stub-llm	Fixed responses after a set delay; can be told to fail	FastAPI	1	None	Real provider used instead if configured; otherwise breaker opens
ingest (one-shot)	Parse, chunk, embed and index corpus; write index_meta.json	Python CLI, sentence-transformers, Chroma	Run once, or on demand	Writes index volume	Workers do not start (dependency); rerun is idempotent
prometheus	Scrapes api, worker, ingest gauges	Prometheus	1	Time series	No metrics; service unaffected
jaeger	Receives OpenTelemetry traces	Jaeger all-in-one	1	In-memory	No traces; service unaffected
grafana (optional)	Dashboard: p50/p95, errors, throughput, availability, CPU/memory, breaker	Grafana, cAdvisor	1	Provisioned	Dashboard unavailable

4. Deployment boundaries and why they are placed there

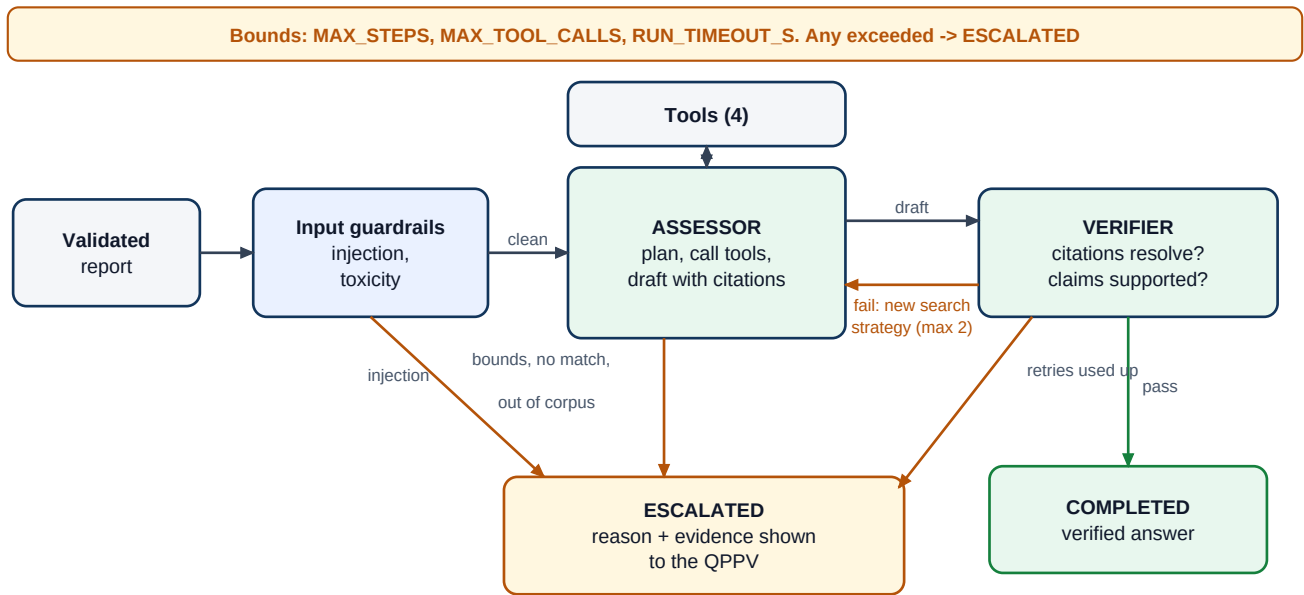
Boundary	Separated from	Reason (scaling, isolation, release, resources)
Edge (api)	worker	Scaling and isolation. Intake must stay responsive while workers are busy for tens of seconds. API is light; worker is CPU and memory heavy. A slow or crashing run cannot block intake.
Execution (worker)	api, redis	Scaling and resource profile. Workers scale independently with --scale worker=3. They hold the embedding model and the agent loop, so memory per worker is the main cost of scaling.
State (redis)	api, worker	Failure isolation. A single durable store for queue and results means either side can restart without losing runs.
Model (stub or provider)	worker	Independent release and test isolation. The provider can change by config; the stub lets load tests measure our system, not rate limits.
Knowledge (ingest + index)	worker	Independent release. Corpus and embedding changes are rebuilt and re-embedded without redeploying the runtime. Workers mount the index read-only.
Observability	everything	Failure isolation. Monitoring outages must not affect runs; they cost nothing on the request path.

5. Request lifecycle

Step	Component	Action
1	api	Receive POST /v1/reports; check rate limit (429 if exceeded)
2	api	Validate schema, types, ranges, categories, cross-field rules. On failure: write to quarantine with field errors, return 422, enqueue nothing
3	api	Normalise the report and compute a hash including prompt version and index build id; on cache hit return the stored result reference
4	api	Create run_id, set state queued, enqueue the job, return 202 with run_id
5	worker	Pick the job, set state running, open the trace, call run_graph(report, run_id)
6	worker	Run input guardrails, then the Assessor and Verifier graph (section 6)
7	worker	Store RunResult and trace; set terminal state completed, escalated or failed; emit metrics and spans
8	client	Poll GET /v1/runs/{run_id} until terminal; optionally GET /trace and POST /feedback

6. Agent architecture

Agent graph - two agents, defined hand-off, bounded loop



Agent	Responsibility	Sees	Produces
Assessor	Plans and calls tools to gather evidence; drafts seriousness, expectedness and routing with citations	Report fields, retrieved passages, tool log	Draft answer with citations (passage_id and quote)
Verifier	Independently checks that each citation resolves to a retrieved passage, each quote appears in it, and the passages support each claim	Draft answer and the retrieved passages only (not the Assessor's reasoning)	Pass, or fail with the unsupported claims listed

Hand-off: the Assessor hands a draft to the Verifier. On fail the Verifier returns the list of unsupported claims and the Assessor searches again with a *different* strategy (for example a rewritten medical term, a clause lookup by id, or a search for a modifying provision). After two failed retries the run escalates.

7. Tools

Tool	Purpose	Notes
rewrite_query(lay_text)	Convert everyday wording into candidate medical terms	LLM-driven with deterministic fallback dictionary; addresses the vocabulary gap
search_corpus(query, k)	Hybrid vector and keyword search over chunks	Returns Passage objects with source, page, clause id, section and score
get_clause(clause_id)	Fetch a clause by identifier	Used to follow references and to find modifying provisions
get_request_fields(run_id)	Read the report's own fields	Timing and outcome come from here, not from free text

Every tool call is logged (tool, input, output summary, latency, ok) and counted against MAX_TOOL_CALLS. Each call has its own timeout.

8. Loop bounds and failure behaviour

Failure	Behaviour
MAX_STEPS, MAX_TOOL_CALLS or RUN_TIMEOUT_S exceeded	Run ends escalated with the bound named in the reason
Assessor crashes mid-run	Run ends failed; job recorded in failed-jobs path; never returned as an answer
Verifier crashes mid-run	Run ends escalated (draft is not returned as confident)

Failure	Behaviour
Worker process killed	Job times out or is picked up as failed by the queue; status failed
Tool error or timeout	Retry within bounds once with a different query; then escalate
Reaction cannot be matched to the insert with confidence	Escalate with the closest passages shown
Timing missing or inconsistent (event before drug start)	Escalate; contract validation catches the obvious cases earlier
Drug outside the corpus	Escalate immediately with reason
Prompt injection detected	Block the field, log, escalate with reason

9. Retrieval design (addresses the vocabulary gap)

- **Parsing:** page-aware; keep clause numbers and section headings as metadata.
- **Chunking:** at clause boundaries, with the heading prepended to each chunk so a fragment keeps its context. For the package insert, an extra set of chunks holds the listed adverse-reaction terms alone, so a medical term matches precisely.
- **Embedding:** all-MiniLM-L6-v2 locally through sentence-transformers; model name from EMBED_MODEL.
- **Bridging lay to medical wording:** rewrite_query produces candidate medical terms; a synonym table in pipeline/synonyms.json expands common lay phrases; search is hybrid (vector plus keyword).
- **Second pass:** after reading the first result, the Assessor re-queries with terms taken from the retrieved text (for example the insert's own term or a cross-referenced clause).
- **Re-embedding:** content hash per chunk; ingest skips unchanged chunks; --reembed rebuilds one document without rebuilding the index.
- **Freshness:** index_meta.json records build time, source versions, chunk count and throughput; also exposed as Prometheus gauges.

10. Data contracts and stored state

Item	Definition
Report (input)	report_id, reporter_type, patient_age (0-120), patient_sex, suspect_drug, drug_start_date, event_date, reaction_description, outcome, narrative. Cross-field: event_date on or after drug_start_date; no future dates. Published as api/contract/report.schema.json
Passage	id, text, source_doc, page, clause_id, section, score (shared/contracts.py)
RunResult	run_id, status (completed escalated failed), routing, routing_basis, seriousness, expectedness, citations, escalation_reason, prompt_versions, tool_calls, trace
Redis keys	rq queue runs; run:{id}:state, :result, :trace; quarantine:{report_id}; cache:run:{hash}; ratelimit:{client}; feedback:{run_id}
Index volume	Chroma persistent directory plus index_meta.json; read-only in workers

11. Reliability and degradation

- **Timeouts:** every model and tool call, plus a whole-run wall-clock limit.
- **Retry:** bounded, exponential backoff with jitter; only for transient errors.
- **Circuit breaker:** closed, open, half-open, per worker process; thresholds BREAKER_FAILS and BREAKER_RESET_S. Trade-off: per-process state avoids adding a shared dependency on the failure path, at the cost of each worker opening independently.
- **Fallback path:** breaker open or provider unavailable, then the run ends escalated with reason 'LLM unavailable'. Never a confident answer.
- **Demonstration:** set STUB_FAIL=true to open the breaker, then clear it to watch recovery.

12. Guardrails

- **Injection defence:** heuristic and pattern checks on every free-text field; user text wrapped in delimiters and treated as data; blocked fields escalate.
- **Output filter:** blocks phrasing that states a final submission or decision; outputs read as recommendations for a qualified person.
- **Toxicity check:** on input and output; limits documented.
- **Tool exposure:** tools take typed arguments; free text never selects a tool or a clause id directly.

13. Observability and traceability

- **Logs:** JSON, with correlation_id equal to run_id in api, worker and each agent step.
- **Traces:** OpenTelemetry, one trace per run, child spans per agent step and tool call, exported to Jaeger.
- **Metrics:** request latency, end-to-end run latency (submission to final result), error rate, throughput, queue depth, runs by status, steps and tool calls per run, retries, breaker state, index freshness, cost.
- **Run reconstruction:** GET /v1/runs/{id}/trace returns every tool call, query, passage, prompt version, per-step latency, tokens and cost, and the verification outcome, persisted so it survives worker restarts.

14. Configuration and secrets

All environment-specific values come from environment variables listed in .env.example: LLM_PROVIDER, LLM_API_KEY, LLM_MODEL, LLM_BASE_URL, REDIS_URL, CHROMA_DIR, EMBED_MODEL, PROMPT_VERSION, OTLP_ENDPOINT, STUB_DELAY_MS, STUB_FAIL, MAX_STEPS, MAX_TOOL_CALLS, RUN_TIMEOUT_S, RATE_LIMIT_PER_MIN, price and breaker settings. .env is git-ignored and a pre-commit scan blocks key-like strings. Prompts are versioned files; the version is stored with every result.

15. Performance and cost hypotheses (to be measured)

Topic	Hypothesis	How it is checked
Per-worker throughput	About 1 / (LLM calls per run x stub delay + retrieval time); scaling to 3 workers raises throughput close to linearly until another component saturates	Locust with --scale worker=1 vs 3
First bottleneck	Worker CPU for embedding the query and vector search, or Redis, or the polling load from clients	Container CPU and Redis stats during the test; vary poll interval
Memory	Each worker loads the embedding model, so memory scales with worker count	docker stats
Caching	Whole-run cache cuts p95 and cost sharply for repeated reports; key includes prompt version and index build id so stale answers are not served	Repeat-request Locust mix
Largest cost driver	Assessor turns, because retrieved passages accumulate in its context	Per-step token accounting; agents/COST.md

16. Decisions and rejected alternatives

Decision	Chosen	Rejected and why
Interaction style	Async: 202 plus polling	Synchronous REST (holds connections for tens of seconds); webhook callback (caller needs an endpoint; kept as optional extension)
Task runner	RQ on Redis	Celery (heavier configuration for no benefit at this scale)
Agent framework	LangGraph	CrewAI (role abstractions hide the loop control and bounds needed for the verifier)
Two agents	Separate Assessor and Verifier with separate prompts and state views	Two prompts in one function (not two agents by the brief's definition; verifier would share the author's bias)
Vector store	Chroma persistent	FAISS (needs a sidecar for metadata and full rebuild for updates; harder to re-embed one document)
Embeddings	Local all-MiniLM-L6-v2	Paid embedding API (the only real cost in the project)

Decision	Chosen	Rejected and why
State store	Single Redis with append-only file	Adding Postgres or SQLite services (more moving parts for a one-day build)
Retrieval	Hybrid with query rewrite and second pass	Single embedding search (the brief says it misses the reference case)
Breaker scope	Per worker process	Shared breaker in Redis (adds a dependency on the failing path)
Load test target	Stub LLM	Real provider (would measure rate limits, not our system)

17. Repository layout

Folder	Owner	Contents
api/, worker/, stub_llm/, infra/, loadtest/, shared/, scripts/	Member A	Service code, compose, Prometheus config, Locust, contracts, helper scripts
agents/, llm/, prompts/, guardrails/	Member B	LangGraph agents and tools, LLM client and resilience, versioned prompts, guardrails
pipeline/, corpus/, eval/, docs/	Member C	Ingestion and retrieval, source PDFs, eval set and runner, PRD, architecture, runbook, incident log, report

PART 3 - Repo addendum prompt

Repo addendum: install PRD and architecture in the repository

Give Member A this after Prompt 0. Put PRD.md and ARCHITECTURE.md in the repo root first.

```
PRD AND ARCHITECTURE ADDENDUM (paste after Prompt 0 finishes; Member A runs it)
GOAL: make the PRD and architecture the shared source of truth in the repo.
1. Copy PRD.md and ARCHITECTURE.md into docs/ (they are provided in the workspace root).
2. Add to AGENTS.md: "Before starting any phase, read docs/PRD.md and docs/ARCHITECTURE.md.
   If your work would contradict them, stop and report the conflict instead of changing them."
3. Verify the folder ownership table, service list, agent topology and data contracts in
   ARCHITECTURE.md match shared/contracts.py and docker-compose plans. List any mismatch.
4. Commit on main: "docs: add PRD and architecture". Print the diff summary.
STOP.
```